

Connexions série



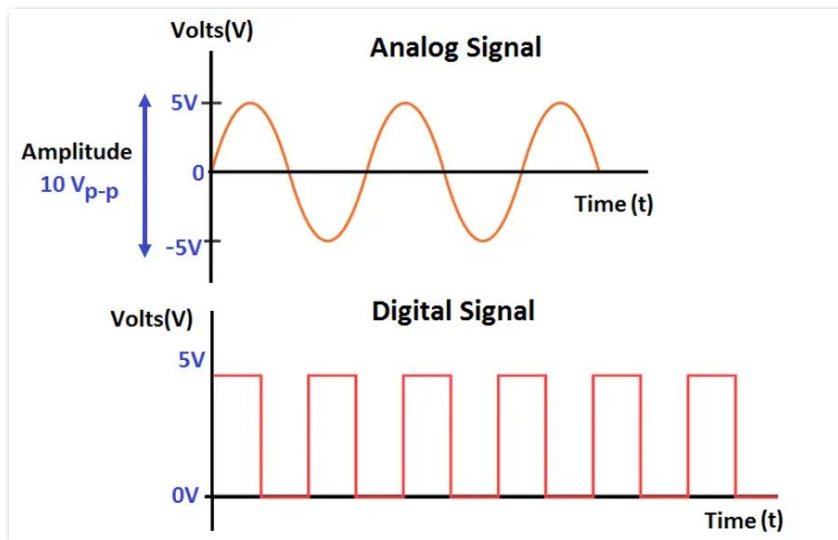


La transmissió analògica vs. digital

Diferència entre transmissió **analògica** i **digital**.

La transmissió analògica transmet valors **continus** (el voltatge pot variar i prendre qualsevol valor (en l'exemple, entre 5v i -5v).

La transmissió digital transmet valors **discrets** (el voltatge només pot ser 5v o 0v), no hi ha terme mitjà. Tradicionalment, s'associa 5v = 1 binari i 0v = 0 binari.





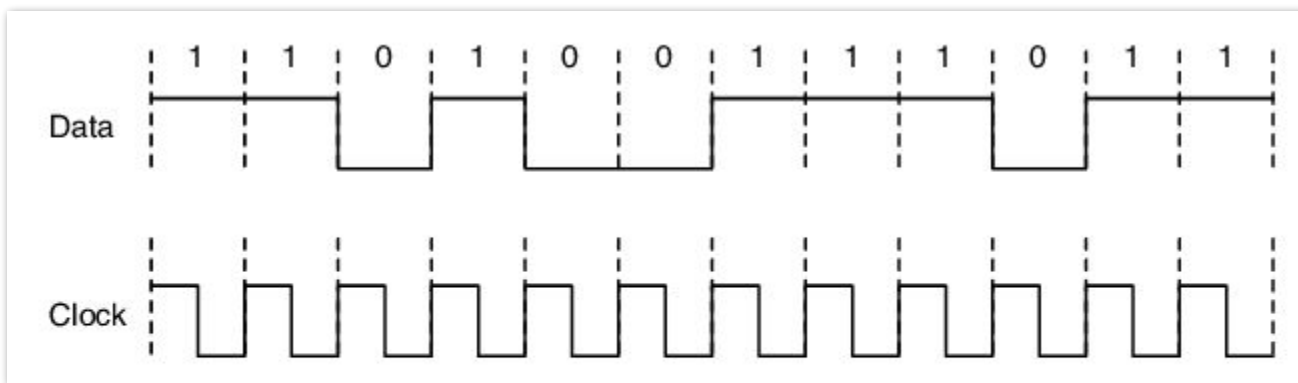
La transmissió digital

La necessitat de sincronitzar:

En l'exemple, l'**emissor** envia la línia Data: 110100111011. Com podem estar segurs que el **receptor** rep aquests valors i no: 111100110000111110011111 ? (que és bàsicament duplicar cadascun dels bits anteriors).

Necessitem un senyal constant (Clock) que li marca al **receptor**, quan ha de llegir el valor (5v o 0v) i interpretar un 0 o 1 binari. La contrapartida és que necessitem un segon cable que porti el senyal del rellotge (CLK)

Bé, hi ha maneres de fer-ho sense rellotge. Veurem el port sèrie, i també podeu buscar com funciona el protocol Ethernet.





Arduino-Transmissio digital.

Per a que hi hagi comunicació digital entre dos dispositius, hi ha diverses maneres de connectar-los:

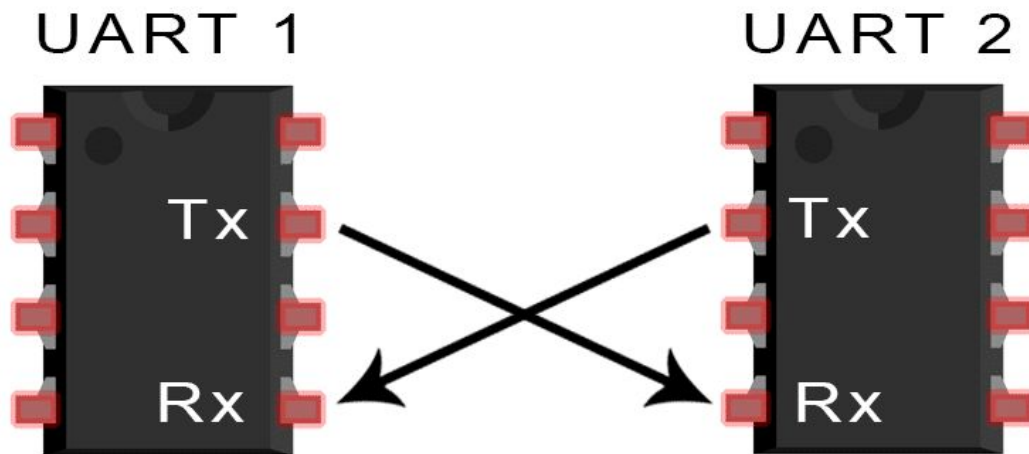
- Sèrie
- I2C
- SPI



Port Sèrie (si, el serial de l'arduino)

Port Sèrie

Exemple de connexió. Cal buscar els pins corresponents al Tx i Rx.





Port Sèrie

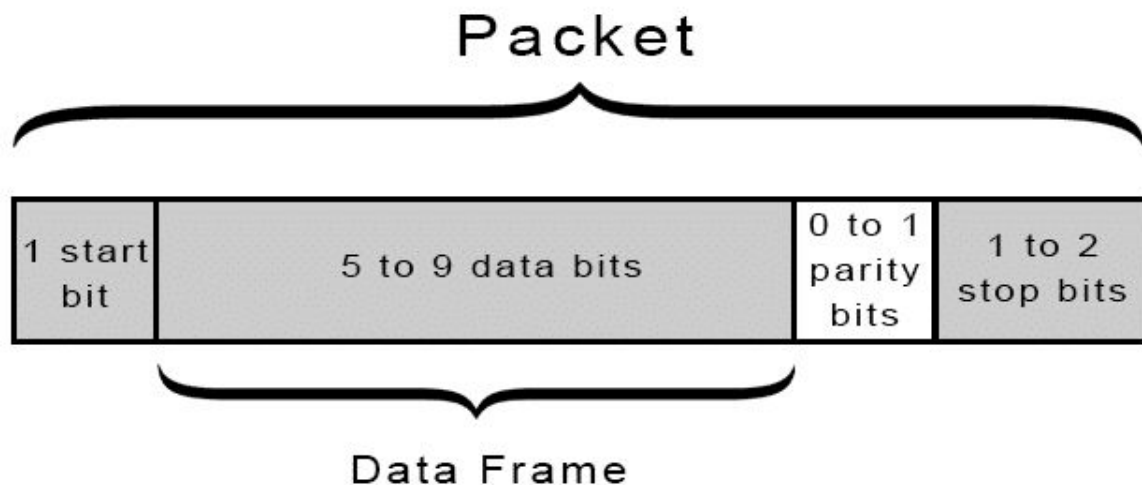
El protocol UART (Universal Asynchronous Receiver-Transmitter) és una interfície de comunicació serial molt utilitzada per enviar i rebre dades entre dispositius.

- **UART no requereix un senyal de rellotge** compartit entre el transmissor i el receptor. Això significa que la sincronització de les dades es basa en l'acord sobre la velocitat de transmissió (baud rate) entre els dispositius.
- El baud rate especifica la velocitat de transmissió de dades, normalment en bits per segon (bps). Tant el transmissor com el receptor **han d'estar configurats amb el mateix baud rate** per a una comunicació correcta. Exemple: 9600 bps, 115200 bps, etc.



Port Sèrie

Les dades transmeses per UART s'organitzen en paquets. Cada paquet conté 1 bit d'inici, de 5 a 9 bits de dades (depenent de l'UART), un bit de paritat opcional, i 1 o 2 bits d'aturada:





Port Sèrie

- **Bits de Dades**

Les dades es transmeten en paquets de bits. Normalment, un paquet UART conté 7, 8 o 9 bits de dades, depenent de la configuració de cada dispositiu.

- **Bit d'Inici i Bit de Parada**

El transmissor afegeix un bit d'inici (start bit), que és un senyal baix (0), per indicar l'inici d'una transmissió. A més, després dels bits de dades, s'afegeix un o més bits de parada (stop bits), que són senyals alts (1), per indicar el final de la transmissió.

- **Bit de Paritat (Opcional)**

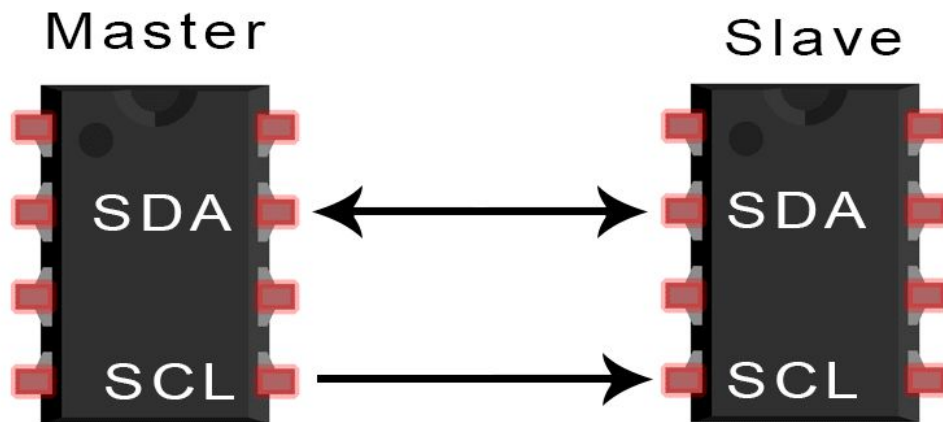
Un bit de paritat pot ser afegit per proporcionar un mecanisme simple de detecció d'errors. Pot ser parell (el nombre de 1's en el paquet serà parell) o senar (el nombre de 1's serà senar). No tots els sistemes usen el bit de paritat.



Port I2C

Port I2C

Exemple de connexió. Cal buscar els pins corresponents al SDA i el SCL de cada dispositiu





Port I2C

- **I2C** (Inter-Integrated Circuit, també conegut amb el nom de TWI -“Two-wire”, “dos cables” –): és un sistema molt utilitzat a la indústria principalment per comunicar circuits integrats entre si.
- La seva principal característica és que utilitza dues línies per transmetre la informació:
 - **Dades:** (SDA) serveix per transferir les **dades** (els 0s i els 1s).
 - **Senyal:** (SCL o clock) serveix per enviar el senyal de rellotge.



Port I2C

- Cada dispositiu connectat al bus I²C té una adreça única que l'identifica respecte a la resta de dispositius, i pot estar configurat com a “mestre” o com a “esclau”.
- Un dispositiu mestre és el que inicia la transmissió de dades i a més genera el senyal de rellotge,
- No cal que el mestre sigui sempre el mateix dispositiu: aquesta característica se la poden anar intercanviant ordenadament els dispositius que tinguin aquesta capacitat.



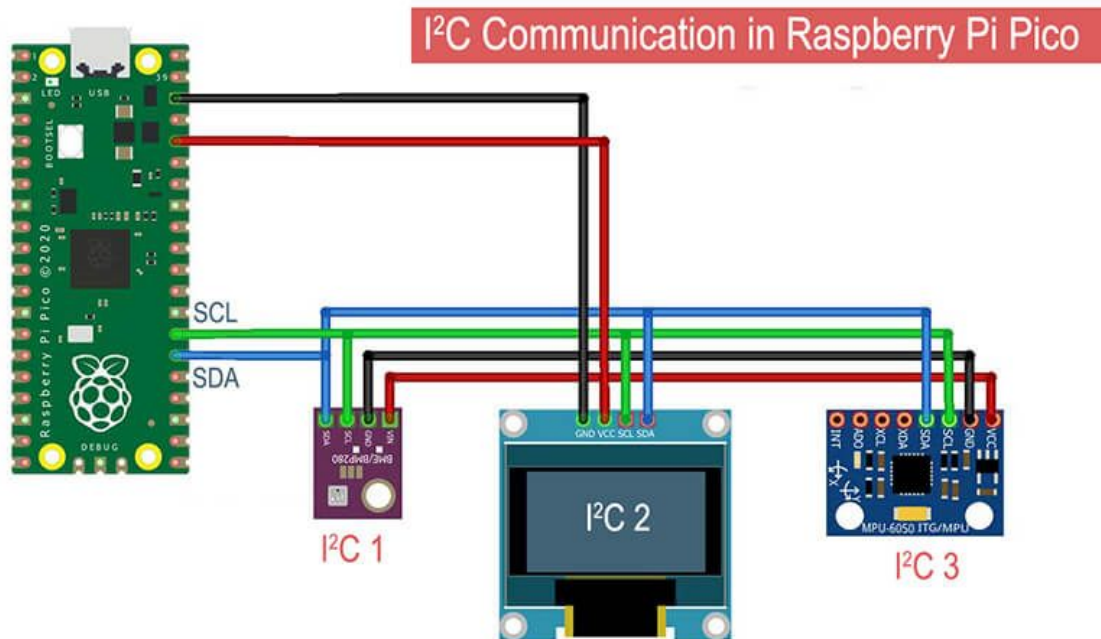
Port I2C

- La velocitat de transferència de dades és de 100 Kbits /s en el mode estàndard (encara que també es permeten velocitats de fins a 3,4 Mbit/s).
- Una única línia de dades, la transmissió d'informació és “**half duplex**”, de manera que en el moment que un dispositiu comenci a rebre un missatge, haurà d'esperar que l'emissor deixi de transmetre per poder respondre-li.



Port I2C

Connexió entre una raspi “Master” i 3 dispositius I2C “Slave” utilitzant només 2 línies de comunicació entre master i slaves.

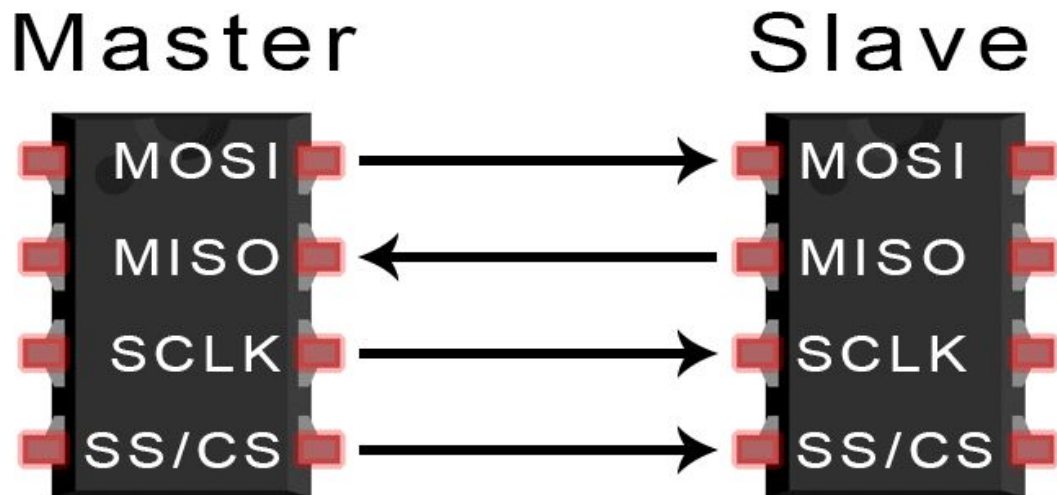




Port SPI

Port SPI

Exemple de connexió. Cal buscar els pins corresponents al SCLK, MOSI, MISO i SS





Port SPI

- **SPI** (Serial Peripheral Interface): Al igual que el sistema I²C, el sistema de comunicació SPI permet controlar (a curtes distàncies) qualsevol dispositiu electrònic digital que accepti un flux de bits sèrie sincronitzat (és a dir, regulat per un rellotge).
- Un dispositiu connectat al bus SPI pot ser “mestre” –en anglès, “màster” – o “esclau”.
- Com amb I²C , amb SPI tampoc cal que el mestre sigui sempre el mateix dispositiu i el segon es limita a respondre.



- La diferència més gran entre el protocol SPI i l'I²C és que el primer requereix de quatre línies (“cables”) en comptes de dues.
 - Una línia “**SCK** o **CLOCK**” envia a tots els dispositius el senyal de rellotge generat pel mestre actual.
 - Línia “**SS**” és la utilitzada per aquest mestre per triar en cada moment amb quin dispositiu esclau vol comunicar-se d'entre els que estan connectats.
 - Línia “**MOSI**” és la línia utilitzada per enviar les dades –0s i 1s– des del mestre cap a l'esclau escollit.
 - i l'altra “**MIS**” és la utilitzada per enviar les dades en sentit contrari: la resposta d'aquest esclau al mestre.

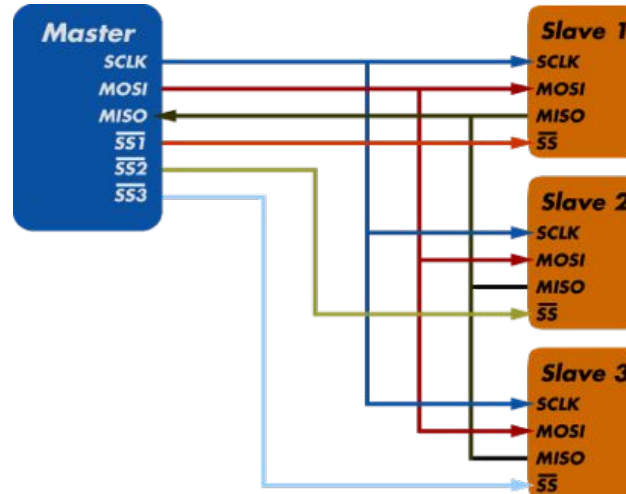


- Es fàcil veure que, en haver-hi dues línies per a les dades, la transmissió d'informació és “**full duplex**” (és a dir, que la informació pot ser transportada en ambdós sentits alhora).
- Quan hi ha més d'un esclau s'utilitza una línia “SS” diferent per a cadascun, serveix per activar l'esclau concret que en cada moment el mestre vulgui utilitzar
- Les línies, “MOSI” i “MISO”, són compartides per tots els dispositius).



Port SPI

Exemple connexió diferents dispositius. Cal una línia SS addicional per cada dispositiu





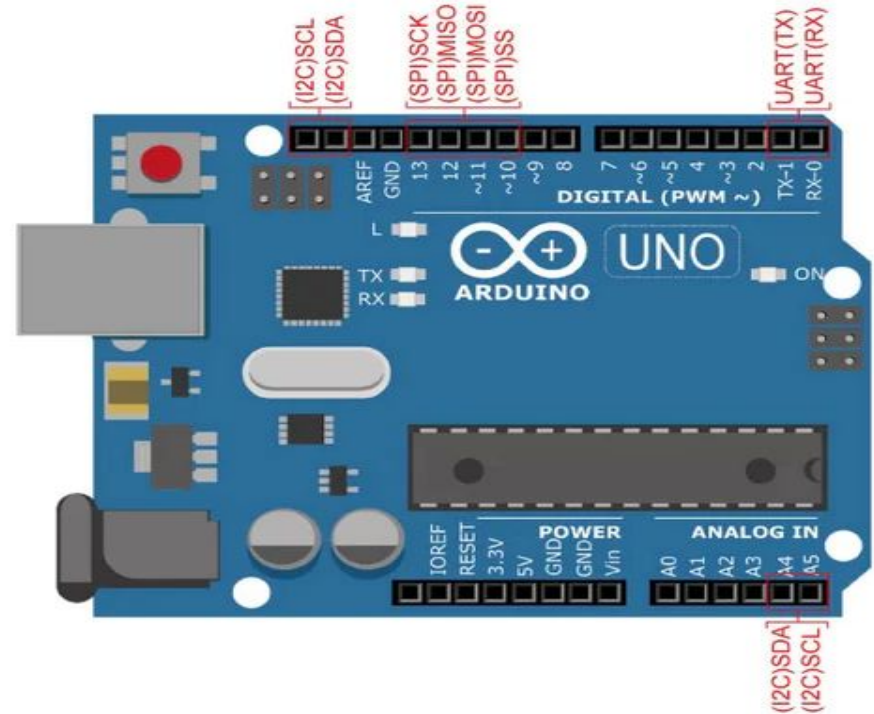
Tècnicament parlant, l'esclau que rebi per la seva línia SS un valor de voltatge BAIX serà el que estigui seleccionat en aquest moment pel mestre, i els que rebin el valor ALT no ho estaran (d'aquí el subratllat superior que apareix a la figura).

Com podeu veure, el protocol SPI respecte de l'I²C, exigeix dedicar més pins d'E/S a la comunicació externa. Com a avantatge, és més ràpid i consumeix menys energia que I²C.



Connexió arduino. Serie, i2c i spi

- Els pins 0 i 1 estan dedicats al port sèrie (Rx i Tx respectivament)
- Els pins corresponents a les línies I2C:SDA i SCL són els números 16 i 17, i
- Els pins corresponents a les línies SPI: SS, MOSI, MISSO i SCK són els números 13, 12, 11 i 10.
- Si es necessiten més línies SS es podrà utilitzar qualsevol altre pin d'E/S. Recordeu!! S'ha de posar el valor del seu voltatge de sortida a BAIX quan es vulgui treballar amb el dispositiu esclau associat i posar a ALT la resta de pins SS.





Treball amb connexions Serie

Activitats

- Crea un esquema amb tinkercad amb un lcd i2c connectat. Executa un programa de prova que trobaràs a l'apartat Starters del panell de components.
- Afegeix la pantalla LCD a l'esquema de l'exercici 3 (botons i led), per a que mostri textos diferents en funció de si el botó esta pulsat o no pulsat. (que mostri el text "On" o "Off"
- Pots crear alguna animació del text mentre el botó està pulsat?